

Machine Learning: Neural Networks and Fully Vectorized Backpropagation

Quoc Kien

1. Nơ-ron Đơn Bào (Perceptron) và Hiện tượng Bảo hòa Gradient

Mô hình nơ-ron cơ bản thực hiện một phép biến đổi tuyến tính kết hợp với hàm kích hoạt phi tuyến:

$$z = \mathbf{w}^T \mathbf{x} + b \implies \hat{y} = a = g(z)$$

Hiện tượng Bảo hòa Gradient (Gradient Saturation)

Khi sử dụng hàm kích hoạt Sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$, đạo hàm của nó là $\sigma'(z) = \sigma(z)(1 - \sigma(z))$. Khi giá trị z tiến về cực hạn ($z \rightarrow \infty$ hoặc $z \rightarrow -\infty$), giá trị của $\sigma(z)$ tiến sát về 1 hoặc 0, dẫn đến đạo hàm $\sigma'(z) \rightarrow 0$. Khi thực hiện lan truyền ngược qua nhiều tầng ẩn, các đạo hàm tiệm cận 0 này nhân chuỗi với nhau sẽ làm gradient triệt tiêu hoàn toàn về 0. Mạng nơ-ron rơi vào trạng thái bảo hòa và dừng học (**Vanishing Gradient Problem**).

Để khắc phục, các tầng ẩn hiện đại ưu tiên sử dụng hàm **ReLU** $g(z) = \max(0, z)$ vì đạo hàm bằng 1 với mọi miền dương, hoặc **Leaky ReLU** để tránh hiện tượng nơ-ron chết:

$$g(z) = \max(\alpha z, z) \implies g'(z) = \begin{cases} 1 & \text{nếu } z > 0 \\ \alpha & \text{nếu } z \leq 0 \end{cases} \quad (\text{với } \alpha \approx 0.01)$$

The plot shows why deep networks often avoid sigmoid activations in hidden layers: once z is far from zero, the sigmoid slope is nearly flat, so backpropagation sends almost no correction signal to earlier layers.

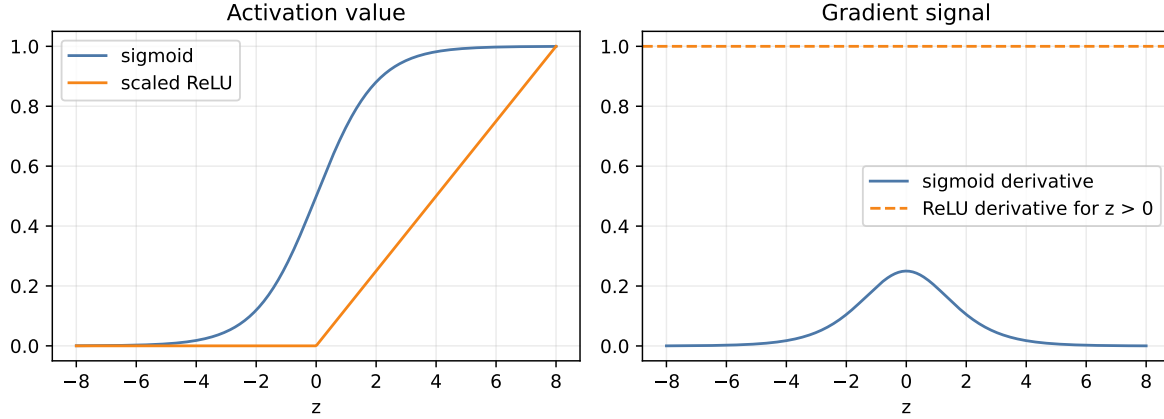


Figure 1: Sigmoid saturates for large positive or negative inputs, while ReLU preserves a usable gradient on the positive side.

2. Kiến trúc Mạng Nơ-ron Đa Tầng (Multilayer Perceptron - MLP)

The easiest way to read an MLP is layer by layer. Each circle stores one scalar activation, and each edge has a learnable weight. A fully connected layer means every activation from the previous layer contributes to every neuron in the next layer.

C:\Users\Quoc Kien\AppData\Local\Temp\ipykernel_1904\821392431.py:13: UserWarning: Setting the circle = plt.Circle((layer_idx, y), 0.14, color="#4C78A8", edgecolor="black", linewidth=0.5)

This diagram also clarifies the matrix notation below: multiplying by $\mathbf{W}^{[l]}$ collects all incoming edges for layer l , adding $\mathbf{b}^{[l]}$ shifts each neuron, and applying $g^{[l]}$ turns the linear score into the activation passed forward.

Để xử lý các bài toán phi tuyến phức tạp, các nơ-ron được xếp chồng thành kiến trúc đa tầng đầy đủ kết nối (Fully Connected). Quá trình lan truyền xuôi (Forward Pass) cho một Mini-batch gồm m mẫu dữ liệu được biểu diễn đồng thời dưới dạng ma trận để tối ưu hóa hiệu năng tính toán:

$$\mathbf{Z}^{[l]} = \mathbf{W}^{[l]} \mathbf{A}^{[l-1]} + \mathbf{b}^{[l]}$$

$$\mathbf{A}^{[l]} = g^{[l]}(\mathbf{Z}^{[l]})$$

Kích thước của các ma trận trong hệ thống (với n_l là số nơ-ron tại layer l): * $\mathbf{A}^{[l-1]}$: Kích thước $(n_{l-1} \times m)$ — với $\mathbf{A}^{[0]} = \mathbf{X}^T$ là ma trận đầu vào. * $\mathbf{W}^{[l]}$: Kích thước $(n_l \times n_{l-1})$ — ma

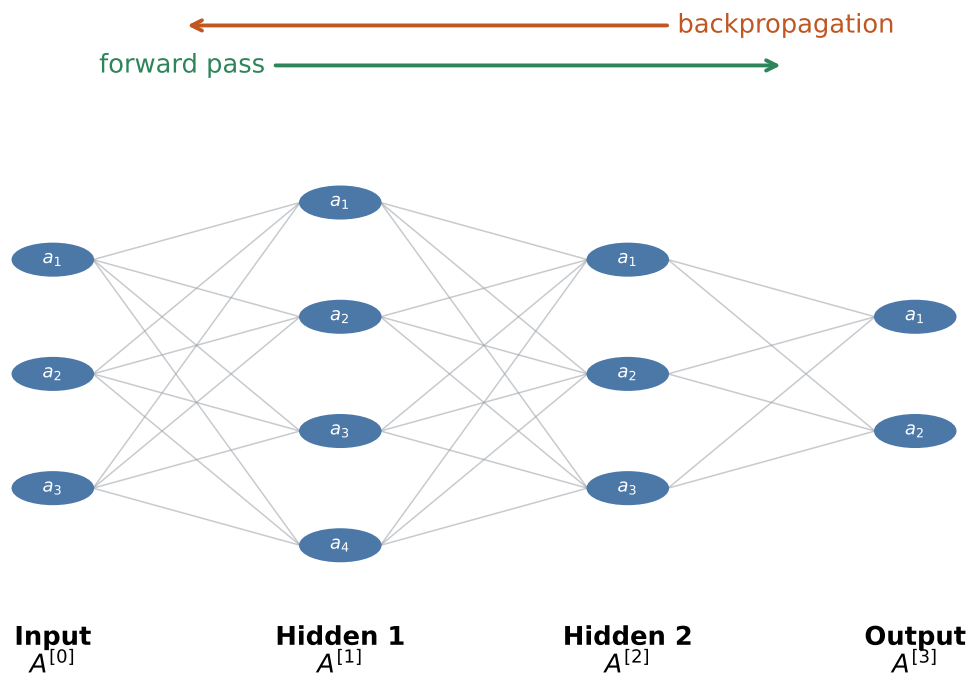


Figure 2: A fully connected neural network with input, hidden, and output layers.

trận trọng số của tầng l . * $\mathbf{b}^{[l]}$: Kích thước $(n_l \times 1)$ — vector bias (được cộng broadcast theo cột cho đủ kích thước batch m). * $\mathbf{Z}^{[l]}$ và $\mathbf{A}^{[l]}$: Kích thước $(n_l \times m)$.

3. Chứng Minh Toán Học Đạo Hàm Ma Trận Lan Truyền Ngược (Vectorized Backpropagation)

Xét hàm mất mát lỗi bình phương tổng quát hoặc Cross-Entropy viết cho toàn bộ một batch gồm m mẫu dữ liệu: $\mathcal{L}$. Ta định nghĩa ma trận lỗi hệ thống tại tầng l bằng đạo hàm riêng của Loss đối với tổng tuyến tính ma trận $\mathbf{Z}^{[l]}$:

$$\mathbf{dZ}^{[l]} = \frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{[l]}} \quad (\text{kích thước } n_l \times m)$$

Sử dụng toán tử tích Hadamard $\odot$ (element-wise product), ta tiến hành chứng minh chuỗi công thức vi phân ma trận ngược từ tầng cuối cùng L về các tầng ẩn trước đó.

Bước 1: Gradient tại tầng đầu ra (Output Layer $l = L$)

Giả sử bài toán phân loại đa lớp sử dụng hàm kích hoạt Softmax và hàm mất mát Cross-Entropy ở tầng cuối. Triển khai vi phân chuỗi ma trận và triệt tiêu các lượng mẫu số tương ứng, ta thu được ma trận sai số đầu ra cực kỳ clean:

$$\mathbf{dZ}^{[L]} = \mathbf{A}^{[L]} - \mathbf{Y}$$

Bước 2: Lan truyền ngược ma trận sai số về tầng ẩn l

Ta cần tính $\mathbf{dZ}^{[l]}$ dựa trên ma trận sai số của tầng liền sau nó $\mathbf{dZ}^{[l+1]}$. Áp dụng quy tắc vi phân chuỗi tổng quát trên không gian đa biến:

$$\mathbf{dZ}^{[l]} = \frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{[l]}} = \left[\frac{\partial \mathbf{Z}^{[l+1]}}{\partial \mathbf{A}^{[l]}} \right]^T \frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{[l+1]}} \odot (g^{[l]})'(\mathbf{Z}^{[l]})$$

Vì phương trình tầng kế tiếp là $\mathbf{Z}^{[l+1]} = \mathbf{W}^{[l+1]}\mathbf{A}^{[l]} + \mathbf{b}^{[l+1]}$, đạo hàm riêng $\frac{\partial \mathbf{Z}^{[l+1]}}{\partial \mathbf{A}^{[l]}} = \mathbf{W}^{[l+1]}$. Thay thế vào hệ thống, ta thu được phương trình lan truyền lỗi cốt lõi:

$$\mathbf{dZ}^{[l]} = \left((\mathbf{W}^{[l+1]})^T \mathbf{dZ}^{[l+1]} \right) \odot (g^{[l]})'(\mathbf{Z}^{[l]})$$

Bước 3: Tính Gradient đối với các tham số Ma trận $\mathbf{W}^{[l]}$ và $\mathbf{b}^{[l]}$

Từ ma trận lỗi $\mathbf{dZ}^{[l]}$, ta tính tốc độ thay đổi của hàm mất mát đối với các tham số cần tối ưu hóa của tầng đó (chia cho m để lấy trung bình mẫu trên toàn bộ mini-batch):

$$\mathbf{dW}^{[l]} = \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[l]}} = \frac{1}{m} \mathbf{dZ}^{[l]} (\mathbf{A}^{[l-1]})^T \quad (\text{kích thước } n_l \times n_{l-1})$$

$$\mathbf{db}^{[l]} = \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[l]}} = \frac{1}{m} \sum_{i=1}^m \mathbf{dZ}^{(i)[l]} = \frac{1}{m} \text{np.sum}(\mathbf{dZ}^{[l]}, \text{axis} = 1, \text{keepdims} = \text{True})$$

4. Các Kỹ Thuật Điều Hướng Chống Overfitting Hiện Đại

4.1 Cơ chế Ngắt Kết Nối Ngẫu Nhiên (Dropout)

Tại mỗi bước lặp huấn luyện (training iteration), thuật toán tiến hành ngắt kết nối ngẫu nhiên một tỷ lệ p các nơ-ron trong tầng ẩn bằng cách đặt giá trị output của chúng bằng 0.

Về mặt toán học, phép toán này tương đương với việc bọc một ma trận mặt nạ nhị phân Bernoulli $\mathbf{M}^{[l]}$ vào quá trình lan truyền:

$$\mathbf{A}^{[l]} = g^{[l]}(\mathbf{Z}^{[l]}) \odot \mathbf{M}^{[l]} \quad \text{với } M_{j,k}^{[l]} \sim \text{Bernoulli}(1 - p)$$

Khi kiểm thử (Testing), ta không dùng Dropout mà giữ nguyên toàn bộ mạng, nhưng phải nhân kết quả đầu ra với hệ số tỷ lệ $(1 - p)$ để đảm bảo kỳ vọng toán học của các tầng ổn định (hoặc áp dụng Inverted Dropout ngay trong lúc train bằng cách chia trực tiếp $\mathbf{A}^{[l]}$ cho $1 - p$). Dropout ép mạng nơ-ron không được phụ thuộc vào bất kỳ một liên kết đơn lẻ nào, tăng tính tổng quát hóa tương tự như mô hình tập hợp (Ensemble).

4.2 Chuẩn hóa Lớp Dữ liệu (Batch Normalization)

Để giải quyết hiện tượng dịch chuyển phân phối nội bộ phức tạp (**Internal Covariate Shift**) khiến các tầng ẩn sâu rất khó hội tụ, ta thực hiện chuẩn hóa phân phối đầu ra tuyến tính $\mathbf{Z}^{[l]}$ về trạng thái trung bình bằng 0 và phương sai bằng 1 trên mỗi mini-batch dữ liệu $\mathcal{B}$:

$$\mu_{\mathcal{B}} = \frac{1}{m} \sum_{i=1}^m \mathbf{z}^{(i)}, \quad \sigma_{\mathcal{B}}^2 = \frac{1}{m} \sum_{i=1}^m (\mathbf{z}^{(i)} - \mu_{\mathcal{B}})^2$$

$$\hat{\mathbf{z}}^{(i)} = \frac{\mathbf{z}^{(i)} - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$$

Để không làm mất đi tính đại diện phi tuyến của tầng dữ liệu, mạng nơ-ron được bổ sung hai tham số có thể học được là γ (scale) và β (shift) để biến đổi lại:

$$\tilde{\mathbf{z}}^{(i)} = \gamma \odot \hat{\mathbf{z}}^{(i)} + \beta$$

Giá trị $\tilde{\mathbf{z}}^{(i)}$ này mới là giá trị thực tế được đưa vào hàm kích hoạt $g(\cdot)$. Batch Normalization hoạt động như một chất bôi trơn toán học mạnh mẽ, cho phép mạng sử dụng learning rate lớn hơn và tăng tốc độ hội tụ lên gấp nhiều lần.